PanWatch（盯盘侠）架构分析报告

仓库：<https://github.com/TNT-Likely/PanWatch>（main 分支，分析时点 2026-10-24）

分析方式：纯网络读取（GitHub API / raw），未 clone 仓库；由多 AI 队友并行协作完成

配套子报告：backend-analysis.md (backend-analysis.md)（后端）· frontend-analysis.md (frontend-analysis.md)（前端）· agent-flow.md (agent-flow.md)（Agent 编排与数据流）

版本：综合报告最终完整版（三份子报告均已回填；backend 子报告的 9 项「待补充」已基于实测源码补全，GitHub 匿名 API 限速窗口于 23:33 重置后抓取完成）

---

0. 一句话总结

PanWatch（盯盘侠） 是一个自托管的 AI 股票盯盘平台，覆盖 A 股 / 港股 / 美股，用多智能体（TradingAgents 体系）做持仓分析、实时提醒与自动报告，提供可安装到手机主屏的 PWA 前端与 FastAPI 后端，开箱即用地把"行情获取 → AI 分析 → 报告生成 → 多渠道通知"串成一条流水线。

---

1. 技术栈总览

层	技术	说明
后端	Python 3.11+ / FastAPI / APScheduler / SQLAlchemy(SQLite)	模块化单体（modular monolith），单进程单库
AI 编排	LangGraph（自研 `pan-investment-research`）+ 上游 `TradingAgents v0.7.0`	两套多智能体系统并存
数据源	自研多供应商路由（腾讯/东财/新浪/雪球/Stooq/Yahoo/YFinance/同花顺/财联社）+ akshare	覆盖行情/ K 线/基本面/新闻/资金流/快讯
前端	React 18 + TypeScript + Vite + Tailwind + shadcn/ui 风格	pnpm monorepo，PWA 可安装

可观测性	OpenTelemetry（默认 no-op，配 OTLP 才导出）	trace_id 贯穿 agent_runs 表与 span
部署	Docker / docker-compose / 本地 `make dev-api`	一键 `docker run` 起服务

---

2. 后端架构（模块化单体）

来源：backend-analysis.md (backend-analysis.md)

2.1 整体装配

入口是根目录 `server.py`，但真正的 FastAPI 应用对象在 `src/bootstrap/application.py` 创建并注册全部 router。`server.py` 只做三件事：

1. 进程级初始化：`lifespan()` 完成 DB 初始化、日志、代理/SSL、Playwright、认证种子、Agent 种子、数据源对账、调度器启动；

2. SPA 静态托管：挂载兜底路由返回 `index.html`；

3. 进程启动：`uvicorn.run("server:app", host="0.0.0.0", port=8000)`。

2.2 四层分层（依赖方向 `web → modules → platform`）

```
src/
├── bootstrap/     # 应用启动与依赖装配（唯一创建 FastAPI 的地方）
├── platform/      # 技术能力（persistence / ai / marketdata / events / scheduling / notifications / observability / runtime）—— 不得反向依赖 modules
├── modules/       # 产品业务能力（assistant / automation / market / portfolio / research / strategy / paper_trading / reporting / administration）
└── web/           # 跨模块复用的 HTTP 中间件与响应适配（response/errors/datetime）
```

- platform：描述"怎样连接/执行"（引擎、AI client、行情供应商路由、交易日历、通知通道、OTel、运行时配置）。
- modules：一级目录 = 一个产品能力，推荐 `api.py / service.py / repository.py / models.py / schemas.py` 内部结构；禁止跨模块直接 import 对方的 `models.py`/`repository.py`，须经 service/DTO/event 协作。
- web：仅做 HTTP I/O 映射，不承载 SQL / Agent 循环 / 策略判断。

2.3 关键子系统

- 数据源层：`server.py` 内置 `DATA_SOURCE_SEEDS` 预置 14 类供应商（news/kline/quote/capital_flow/fundamentals/northbound/chart…），按 `priority` 做回退（如 K 线：腾讯→东财→Stooq→Yahoo），支持代理与开关。
- 定时调度：`AgentScheduler`（APScheduler 封装）+ `PriceAlertScheduler`(60s) + `PaperTradingScheduler`(60s) + `ContextMaintenanceScheduler`(cron 04:30) + `prune_mcp_logs`(cron 04:00)。
- 鉴权：Docker 部署时 `init_auth_from_env` 从 `AUTH_USERNAME/AUTH_PASSWORD` 写首个账户；首次访问有 onboarding 引导。
- 持久化：SQLite（`platform/persistence`：engine/Session/Base/ORM/migrations），数据目录由 `get_data_dir()` 决定；全量日志经 `DBLogHandler` 入库供 UI 日志板查询。
- 通知：`NotifierManager` + `NotifyPolicy`（时区/勿扰/重试/去重 TTL），渠道含 Telegram / WeCom / DingTalk / Feishu / Bark / Webhook，四级解析（stock_agent→agent→系统默认）。
- 可观测性：`init_otel()` 仅在配置 `OTEL_EXPORTER_OTLP_ENDPOINT` 且装 OTel SDK 时启用，否则静默 no-op，绝不阻断启动。

---

3. 前端架构（React/TS PWA 工作区）

来源：frontend-analysis.md (frontend-analysis.md)

3.1 技术栈与工程结构

- pnpm monorepo（`pnpm-workspace.yaml`），内部拆 3 个本地包：`packages/api`、`packages/base-ui`、`packages/biz-ui`。
- 构建 Vite + PostCSS + Tailwind；测试 Vitest；组件风格为 shadcn/ui（基于 Radix）。
- 277 个文件，`src/pages/` 下页面规模差异大（最大的 `Stocks.tsx` 138K、`Settings.tsx` 69K，推测重页面做代码分割/懒加载）。

3.2 内部 UI 包三层依赖

```
api（最底层，仅网络/类型，无 UI 依赖）
  ↑
base-ui（原子组件 button/dialog/tabs/toast…，cn.ts 类名合并、datetime.ts 时区工具）
```

↑

biz-ui（业务复合组件：K 线/分析/告警/组合卡片与弹窗，组合 base-ui + api）

↑

src（页面/路由，依赖三者）

3.3 关键能力

- 路由：`main.tsx` → `App.tsx`（全局 Provider + 路由）→ `router/page-loaders.ts`（路由级数据预载，避免白屏）。
- 状态/实时：以 React Context + 自定义 Hook 为主（未见 Redux/Zustand）；`useQuoteRefresh` 是行情实时刷新核心；`NotificationProvider` 形成"Provider 注入 + Hook 消费 + 后端 API"通知链路。
- 国际化：i18next 体系，`en-US` / `zh-CN` 双语 locale 对称；界面语言可运行时切换；时区由用户设置驱动、在格式化层统一处理；双语 PWA manifest。
- PWA：`manifest.json` / `manifest.zh-CN.json` + `sw.js` + 多分辨率图标，满足"可安装"条件后在手机/桌面生成主屏图标，SW 提供应用壳缓存，弱网亦可启动（类原生体验）。
- 与后端通信：`packages/api` 按业务域极细拆分（stocks/dashboard/agents/paper-trading/notifications/sse/chat…）；`client.ts` 统一封装鉴权与错误；`sse.ts` + `chat.ts` 支撑对话流式输出与实时事件。

---

4. Agent 编排与端到端数据流

来源：agent-flow.md (agent-flow.md)

4.1 两套独立多智能体系统（勿混淆）

1. `pan-investment-research`（自研 LangGraph）——驱动三个「会话型」Agent：

- `premarket_outlook`（盘前展望）、`intraday_monitor`（盘中监测）、`daily_report`（收盘复盘）
- 共享同一张强制编译图：`specialist → bull/bear debate → manager → strategy → risk → decision → execution`
- 仅依赖 Pydantic + LangGraph，不连库、不泄露凭证、不下单；宿主注入 model port + 只读 tools + event sink。

2. `TradingAgents`（上游 TauricResearch v0.7.0）——仅用于持仓页「大脑图标」触发的单标的深度分析：

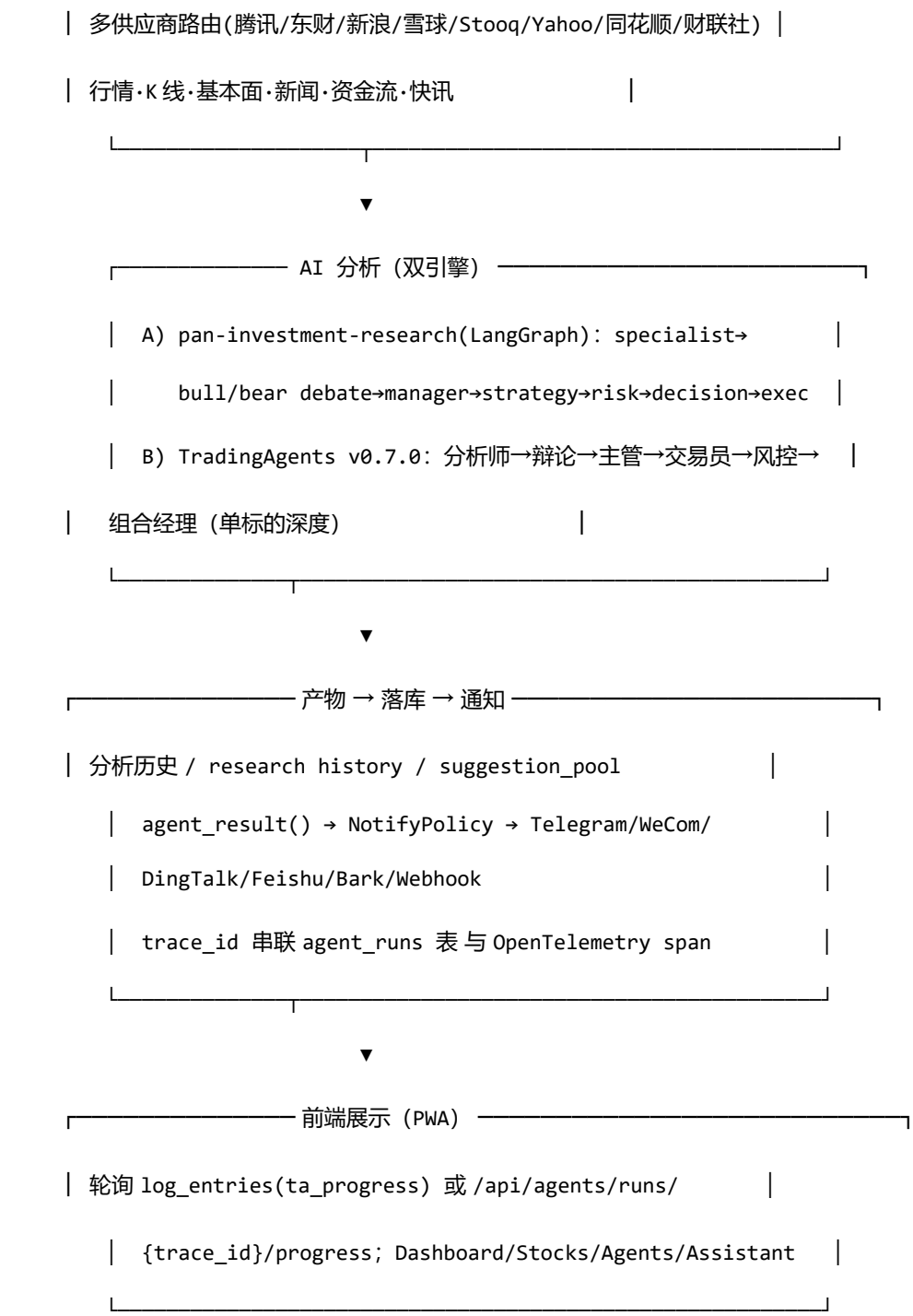
- 流程：`4 类分析师 → 多空辩论 → 研究主管 → 交易员 → 风控 → 组合经理最终决策`
- 评级映射为 买入/增持/持有/减持/卖出/待复核。

4.2 调度与触发

- 统一收敛在 `server.py` + `AgentScheduler`（APScheduler cron/interval）。
- 定时走 cron（结合交易日历过滤），手动走 `trigger_agent_for_stock`；`trace_id` 区分 `schedule/manual/api`，并写 `agent_runs` 生命周期记录。
- `pan-agent-runtime`（独立内核 `pan_agent`）是导航级助手的执行引擎：有界 Agent loop、人工审批、checkpoint、RuntimeEvent 流式事件；与 `pan-investment-research`、TradingAgents 通过「宿主注入 ModelPort/ToolRegistry/EventSink/policy」解耦。

4.3 文字版端到端数据流图




多供应商路由(腾讯/东财/新浪/雪球/Stooq/Yahoo/同花顺/财联社)
行情·K 线·基本面·新闻·资金流·快讯
AI 分析（双引擎）
A) pan-investment-research(LangGraph)：specialist→
bull/bear debate→manager→strategy→risk→decision→exec
B) TradingAgents v0.7.0：分析师→辩论→主管→交易员→风控→
组合经理（单标的深度）
产物 → 落库 → 通知
分析历史 / research history / suggestion_pool
agent_result() → NotifyPolicy → Telegram/WeCom/
DingTalk/Feishu/Bark/Webhook
trace_id 串联 agent_runs 表与 OpenTelemetry span
前端展示（PWA）
轮询 log_entries(ta_progress) 或 /api/agents/runs/
{trace_id}/progress；Dashboard/Stocks/Agents/Assistant

---

5. 部署方式

- 最简（Docker）：`docker run -d --name panwatch -e TZ=Asia/Shanghai -p 8000:8000 -v panwatch_data:/app/data sunxiaoye0721/panwatch:latest`，浏览器开 `http://localhost:8000` 创建登录凭据即可。
- docker-compose：`docker compose up -d`，可改 `TZ` 与卷挂载；注意镜像默认 `Asia/Shanghai` 且不自动检测主机时区，需用 `TZ` 显式声明。
- 本地开发：要求 Python 3.11+ / Node 24 / pnpm 9；`make dev-api`（后端热重载 :8000）+ `make dev-web`（前端 :5183，代理 `/api` → 127.0.0.1:8000）。
- 可观测性导出（可选）：装 OTel 依赖并设 `OTEL_EXPORTER_OTLP_ENDPOINT`，把 trace 发往 Jaeger/Tempo/Langfuse 等；不配置则 no-op。

---

6. 可借鉴的工程亮点

1. 清晰的模块化单体边界：`platform / modules / web / bootstrap` 四层 + 严禁跨模块 import 模型的规则，规模化仍可控。

2. 可注入、可解耦的 AI 内核：`pan_agent` / `pan-investment-research` 都不依赖数据库或 FastAPI，宿主注入 port/registry/sink/policy，便于替换模型与工具。

3. 统一 trace_id 贯穿：`agent_runs` 表 + 日志 + OTel span 同源 trace_id，调试与成本追踪一体化。

4. 强约束的设计系统治理：用 `check-market-colors` / `check-ui-primitives` / `check-i18n-literals` / `check-timezones` 四个 lint 脚本，把涨跌幅配色、UI 原子一致性、文案禁止硬编码、时区处理自动化强制。

5. 数据供应商多源回退：同一数据类型（如 K 线）配多个 provider + priority，单源失效自动降级，对 A/H/US 多市场友好。

---

7. 注意事项与风险

1. 匿名 GitHub API 限速（已克服）：分析初期受未登录 60 次/小时限制（reset 23:33），backend 子报告少数章节曾标「待补充」；限速重置后已通过 Contents API 抓取并本地解

码补齐（application.py 路由全貌、TradingAgents 适配、调度/交易日历、10 类通知渠道、67 张 ORM 表、JWT/PAT、OTel exporter、Makefile/Dockerfile），backend 子报告现已成为完整版。

2. 前端大页面维护成本：`Stocks.tsx`(138K) 与 `Settings.tsx`(69K) 体量偏大，需确认是否已充分拆分/虚拟列表。

3. 自定义状态管理性能：以 Context + hooks 为主，高频 `useQuoteRefresh` 需关注重渲染与性能边界。

4. 时区敏感：调度与通知静默时段依赖正确 `TZ`；Docker 部署务必显式设 `TZ`，否则定时任务与提醒时段会错位。

5. 数据源稳定性：部分源（如 Yahoo/YFinance、北向资金）依赖代理或已断供，生产需按所在网络微调 `enabled` 与代理配置。

---

8. 配套交付物

文件	内容
`backend-analysis.md`	后端：装配、四层职责、TradingAgents 集成、数据源、调度、鉴权、持久化、通知、OTel、部署
`frontend-analysis.md`	前端：技术栈、路由、hooks、i18n、PWA、内部 UI 包三层、前后端通信
`agent-flow.md`	Agent 编排：双多智能体系统、调度/触发、深度分析、助手、通知、OTel、端到端数据流图
`report.md`	本综合报告

说明：三份子报告（backend / frontend / agent-flow）现已全部为完整版，本报告第 2/3/4 节的概览结论已与其一致；backend 的深度细节见 backend-analysis.md (backend-analysis.md) 对应章节。